



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	MJ Programming from First Principles	
Topic Name	User-defined types as a means to model	
Topic Code	-	
Unit/Module No. & Name	10	User-defined types
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4-5
2.0	Meaning And Definition	6-8
3.0	Nature Or Type	9-11
4.0	Examples And Case Studies	12-14
5.0	Keywords And Touch Vocabulary	15



Savitribai Phule Pune University
Centre For Distance and Online Education

OBJECTIVE

1. Understand how user-defined types model real-world objects inside programs.
2. Realize that programs work with rich models, not just numbers or text.
3. Connect system design (boxes, arrows, fields) with code (types, classes, records).
4. Improve correctness and safety by designing types that prevent wrong data.
5. Support reuse by using well-designed types across different parts of a system.
6. Identify where user-defined types are needed in any problem description.
7. Recognize nouns in scenarios (library, clinic, etc.) that can become types.
8. Judge whether a type is too large, too small, or appropriate.
9. Explain and justify type choices clearly in words for effective teamwork.



1.0 INTRODUCTION

1.1 Why We Study Data Types

Every computer program uses data. A calculator uses numbers. A chat app uses messages, names, and time. A music app uses songs, artists, and playlists. When you write code, you must tell the computer what kind of data you want to store and how you want to use it. This is what a data type does.

Most programming languages give you simple or basic types like integer, float, character, string, and Boolean. These are like basic building blocks. They are useful, but they are not enough to show real-life things. For example, a “student” is not only a name. A student has an ID, courses, grades, and contact details.

User-defined types let you create your own custom types. You can group related data together and give it a name. The computer then treats that group like a new type, just like an integer or string. This means you can design your own data shapes for your project.

In beginner programming, students first use only simple types. But when problems grow, variables come in pairs or groups that always stay together. At that point, user-defined types become important. They add structure and make the program easier to understand.

1.1.1 Real World And Programs

There is a strong connection between the real world and programs. When we “model” something, we make a simple version of a real thing that a computer can handle. For example, a map is a model of a city. In the same way, a user-defined type is a model of a real or imaginary thing inside a program.

You cannot store a real person in a computer, but you can store important information about that person. You choose what details to keep and how to organize them. If you plan well, the type becomes simple enough for a computer but detailed enough for your problem. Good modeling is about deciding what to include and what to leave out.

1.2 History Of Data Types

In the early days of programming, languages had only a few basic types. Programmers had to use memory addresses and manual layouts to store complex data. This was fast but very unsafe. A small mistake could crash the program or break important data.

With time, languages added better type systems and clearer ways to make user-defined types. C introduced structures, Pascal introduced records, and Simula (later C++ and Java) introduced

classes. All of these came from the need to store complex data safely. Today, almost all programming languages let you define your own types.

1.3 Data Types In Different Languages

Different languages use different ways to define new types, but the idea is always the same. In C, we use structs. In Java and C#, we use classes and interfaces. In Python, we use classes, and sometimes dictionaries or named tuples.

Functional languages like Haskell and ML use algebraic data types, which combine simple types and variants in a very neat way. Even scripting languages, which are less strict, still follow patterns that act like types. When you learn more languages, you will see that the names change, but the idea of modeling stays the same.



2.0 MEANING AND DEFINITION

2.1 What Is A Data Type

In computer science, a data type is a rule that tells us what kind of value we have and what operations we can perform on that value. An integer type allows addition and subtraction. A string type allows operations like concatenation and searching for characters. The type helps the compiler or interpreter check that you are using values in a safe and meaningful way.

We can think of a type as a label plus a set of allowed actions. When we declare a variable with a type, we are saying, “This box will hold values of this shape, and I will use them in these ways.” If we try to use the variable in a wrong way, the type system can warn us. In this sense, the type system acts as an assistant that reads our code and watches for mistakes.

2.1.1 Primitive Types

Primitive types are built into the language. They are usually implemented in a direct way by the hardware of the computer. Examples include integer types of different sizes, floating point numbers, characters, and Boolean values. They are called “primitive” because they cannot be broken into smaller parts by the type system.

Even though primitive types are simple, they often come with details such as size limits, signed or unsigned modes, and memory layout. Programmers must understand these details when they work close to the hardware. But in most high level programming tasks, primitive types are treated as basic tools that just work and do not need deep thought in every line of code.

2.1.2 User Defined Types

A user defined type is a data type that is created by the programmer, using language features such as structures, classes, records, or enumerations. While primitive types are supplied by the language designers, user defined types come from the needs of a specific problem.

We define a user defined type by giving it a name and describing its internal structure. The structure can be made from primitive types and from other user defined types. Once we have defined such a type, we can declare variables of that type, pass them to functions, return them from methods, and store them in collections.

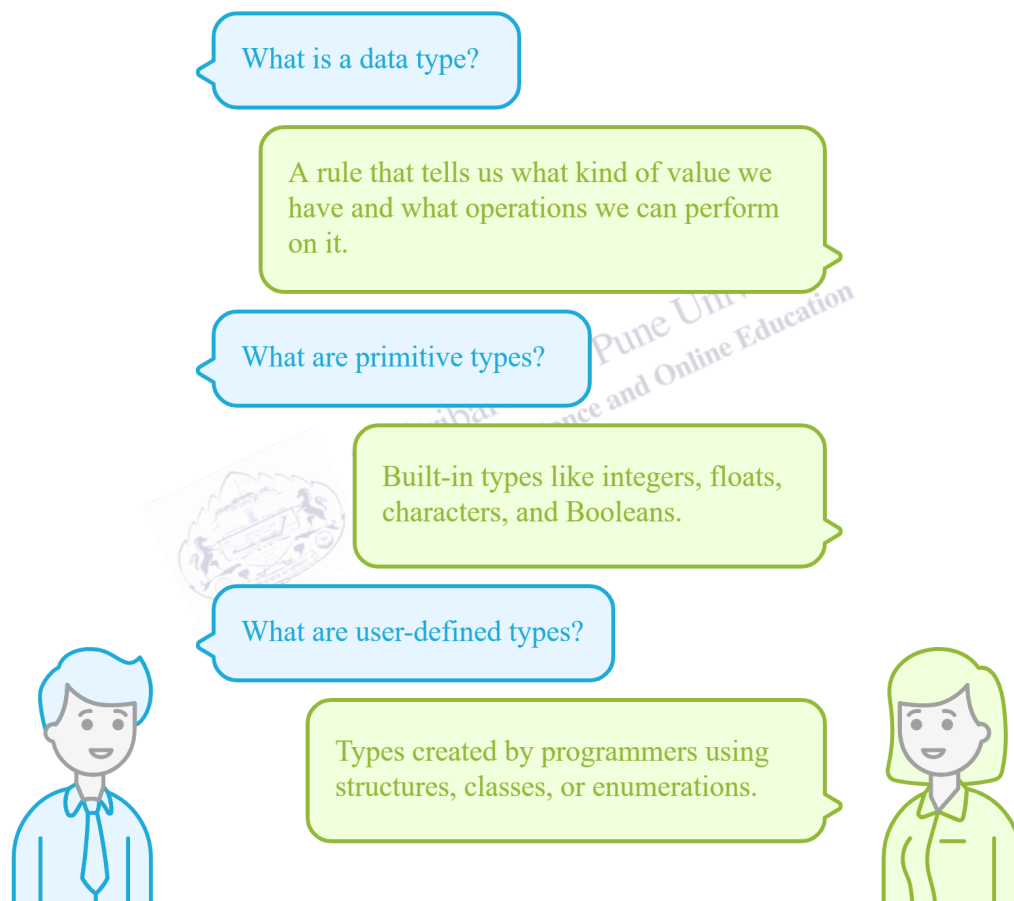
In many languages, user defined types also group behavior with data. For example, a “Song” class in an object oriented language may contain fields such as title, artist, and duration, plus methods such as play, pause, and displayInfo. The type becomes a unit that holds both state and behavior.

2.2 Formal Definition Of User Defined Types

In more formal terms, a user defined type specifies a set of possible values and a set of operations that make sense for those values. The values are built from other types using constructors such as “record of,” “list of,” or “either this or that.” The operations may include reading and writing fields, comparing two values, or applying domain specific rules.

Fig. 1

Data Types Explained



This diagram explains data types in programming. It defines what a data type is, describes primitive types like integers and booleans, and explains user-defined types created by programmers to organize and manage data effectively.

Formal definitions matter when we reason about program correctness. If we know that a certain type always has a nonempty list of items, we can prove that some functions never face an empty input. If we know that a field is an enumeration with only three states, we can check that our code handles all three states and does not forget a case.

2.3 Difference Between Built In And User Defined Types

Built in types are general tools that work in almost every program. User defined types are custom tools that match one family of problems well. Built in types are usually few, stable, and highly optimized. User defined types can be many and flexible, and they change as the design of the system changes.

In practice, we combine both. Primitive types handle low level details such as counts, indices, and simple flags. User defined types handle high level ideas such as Customer, Order, Playlist, or Appointment. This layered use of types mirrors how we think, from concrete numbers to abstract roles.

2.4 Role Of Type Systems

A type system is the set of rules that a language uses to connect types with values and expressions. Some type systems are static, meaning they check most rules before the program runs. Others are dynamic, meaning they check many rules while the program is running. Many modern languages mix both ideas.

For user defined types, the type system helps catch mismatches. If a function expects a Track and you pass a Patient, the compiler can stop you. If a field is declared as an enumeration and you try to store a number outside the allowed set, the system can complain. This protection is a key reason why careful type design is worth the effort.

3.0 NATURE OR TYPE

3.1 Kinds Of User Defined Types

User defined types appear in many forms, depending on the language and the style of programming. The most common forms are records or structs, classes and objects, enumerations, and unions or variant types. Each form has its own nature and is suited to certain modeling tasks.

3.1.1 Records Or Structs

A record, often called a struct in languages like C, is a simple user defined type that groups several named fields. Each field has its own type. All fields are stored together in memory, usually in a fixed layout. Records are good for modeling simple objects that mostly store data and do not need complex behavior.

For example, a record for a student might contain an integer field for the student id, a string field for the name, and another string field for the major. Functions that work on students can take a record as a parameter, but the record itself does not carry methods. This keeps the data and behavior separate, which can be a good match for small systems.

3.1.2 Classes And Objects

A class is a more powerful user defined type that supports both data and methods in one unit. An object is a concrete instance of a class. Classes are central to object oriented programming languages such as Java, C++, C#, and many others. They allow concepts like encapsulation, inheritance, and polymorphism.

When we say that user defined types are a means to model real systems, classes are often what we have in mind. A class can hide its internal details and show only a clean interface. This gives other parts of the program a simple way to interact with the model without needing to know how it is built inside. Well designed classes can also be reused as libraries in many projects.

3.1.3 Enumerations

An enumeration, or enum, is a user defined type that lists a small set of named values. It is useful when a variable can only take one value from a limited list, such as days of the week, levels of access, or states of a process. Enums make code safer and clearer because they avoid magic numbers and repeated string literals.

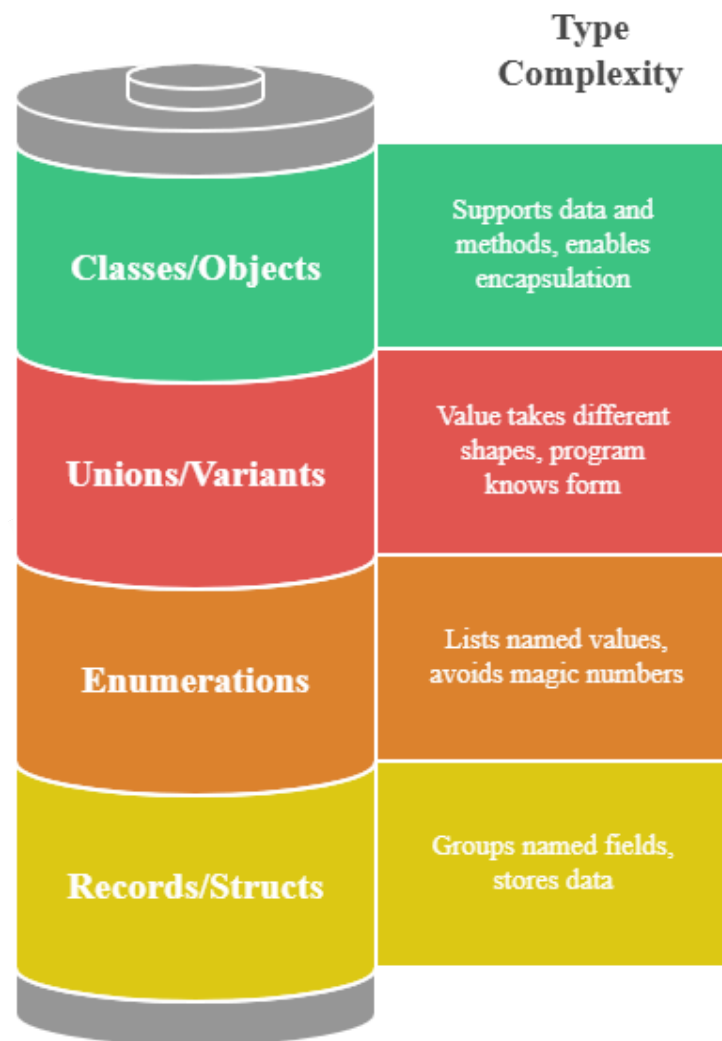
For example, instead of using 0, 1, and 2 to mean “Stopped,” “Playing,” and “Paused,” a music player program can define an enum called `PlayerState` with members `STOPPED`, `PLAYING`, and `PAUSED`. The compiler can then check that only these states are used in the code. This also makes the intent of the code easier for humans to read.

3.1.4 Unions And Variants

A union or variant type allows a value to take one of several different shapes. At any time, the value has one active form, and the program must know which form is in use. Modern languages often provide safe variant types, sometimes called sum types or tagged unions, which carry both the data and a tag that says which case is active.

Fig. 2

User-defined types range from simple data to complex behavior.



This diagram shows different kinds of user-defined data types, arranged by increasing complexity. It explains how records, enumerations, unions, and classes help programmers organize data, represent choices, and combine data with behavior through methods and encapsulation.

Unions are helpful when modeling data that can appear in several related forms. For example, a program that reads media files could define a variant type that can represent an MP3 track, a video clip, or a podcast episode, all under one umbrella type named `MediaItem`. Each case can then have its own fields, such as `resolution` for video or `seriesTitle` for podcasts.

3.2 Choosing The Right User Defined Type

When you design a system, you must choose which form of user defined type fits each part of the model. If you only need to group a few fields, a record may be enough. If you need to attach rich behavior and hide internal details, a class may be better. If a value can only be one of a few named states, an enum is a clear choice. If a value may take several different shapes, a union or variant fits well.

Good designers think about how the types will be used. They avoid making a single giant type with too many fields, because such a type becomes hard to understand and maintain. Instead, they create smaller, focused types that each represent a clear idea.

3.3 Immutable And Mutable Types

Another part of the nature of a type is whether its values can change after creation. A mutable type allows its fields to be updated. An immutable type does not allow such changes; instead, any update creates a new value. Immutable types are common in functional programming and can reduce bugs related to unexpected changes.

For user defined types, you should decide which fields, if any, may change. For example, a `Patient` type might allow the address to change over time, but it should not allow the id or date of birth to change. This decision can be enforced by the language or simply followed as a design rule.

4.0 EXAMPLES AND CASE STUDIES

4.1 Simple Examples

To see how user defined types help us model real situations, let us look at a few simple examples. These examples are written in plain language rather than in a specific coding syntax, so that the ideas remain clear.

4.1.1 Example Of A Student Type

Suppose we want to build a small system to track students in a course. If we try to use only primitive types, we might have separate arrays for student ids, names, and grades. This is hard to manage and easy to break.

Instead, we define a user defined type named Student with fields such as id, name, email, and grade. Now a single Student value keeps related data together. When we pass a Student to a function that prints a report or updates a grade, we know that all needed data travels as a unit.

4.1.2 Example Of A Book Type

In a library system, we can define a Book type with fields like title, author, publicationYear, and isAvailable. We might also attach methods such as issueToMember and returnToShelf if we use classes. Each Book instance then models one real book in the library. The type definition acts as a template for thousands of book records.

4.2 Case Studies

Case studies show a slightly larger picture. They connect user defined types into a small network that models a complete mini system.

4.2.1 Case Study: Modeling A Music Library

Imagine we are designing a simple MP3 music library application. The attached audio material for this unit discusses how user defined types can be used to model MP3 tracks and related concepts. Even without the exact code from that material, we can outline the key models.

We can define a type called Track with fields such as title, artist, album, durationInSeconds, and filePath. Each Track stands for one MP3 file stored on disk. Next, we can define an Artist type with fields like name, country, and a list of tracks. We can also design a Playlist type that has a name and an ordered list of tracks.

By using these user defined types, the music library can answer questions such as “Show all tracks by this artist,” “Play the next track in the playlist,” or “Sort tracks by album.” The types

give structure to the code so that operations become simple loops and method calls rather than confusing movements of raw strings and numbers.

If we choose an object oriented language, we might give the Track class methods such as play, stop, and displayMetadata. The Playlist class might have methods like addTrack and removeTrack. These behaviors stay close to the data they work with, which matches how we think about songs and playlists in our minds.

4.2.2 Case Study: Modeling A Hospital System

Now consider a small hospital information system. We can define a Patient type that stores id, name, dateOfBirth, bloodGroup, and address. A Doctor type may have id, name, specialty, and a list of assigned patients. An Appointment type can connect a patient, a doctor, a date, and a status field that uses an enumeration such as Scheduled, Completed, or Cancelled.

By tying these types together, we get a rich model of the system. When a new appointment is created, the program creates an Appointment value that links to one Patient and one Doctor. When we print a schedule, we loop over Appointment values rather than over separate lists of names and times. Every part of the system talks in terms of these user defined types, which makes the code easier to maintain and extend.

4.3 Comparing The Case Studies

The music library and the hospital system look very different on the surface. One deals with songs and playlists, while the other deals with patients and doctors. Yet, at the level of user defined types, they share many patterns. Both have central types that stand for key actors, such as Track or Patient. Both have linking types, such as Playlist or Appointment, that connect these actors in meaningful ways.

Both systems also use enums to model states, such as PlayerState in the music app or AppointmentStatus in the hospital system. This shows that once you learn certain modeling tricks, you can carry them from one domain to another. The names change, but the structure of the types remains familiar.

4.4 Common Mistakes In Modeling

Beginners often make a few common mistakes when designing user defined types. One mistake is to use only primitive types, even when a new type would make the code clearer. Another mistake is the opposite: creating too many tiny types that do not add meaning, such as wrapping an integer id in a separate type that is never given more structure.

A further mistake is to mix unrelated ideas in one type. For example, a Student type that also stores details about the last exam room and the cafeteria balance may be carrying too much. In

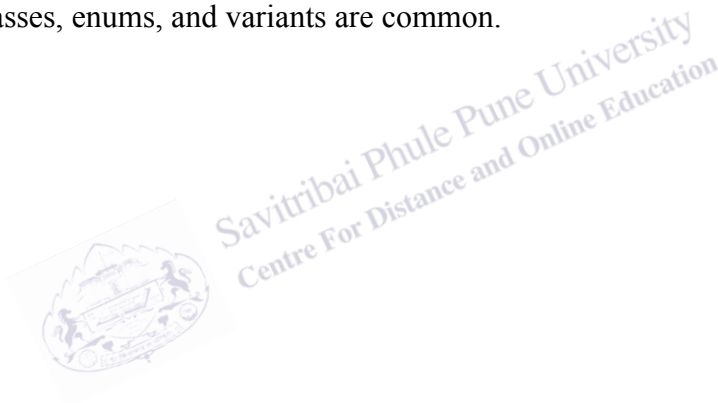
such cases, it is better to create smaller types, such as ExamRecord and PaymentAccount, and link them to the Student type.

4.5 Reflection Activities

To deepen your understanding, you can try simple reflection activities. Take a real system you know, such as your college portal or a shopping app, and list the main nouns that appear on the screen. Try to guess what user defined types the programmers may have created for those nouns. Then think about what fields and methods each type might contain.

You can also rewrite a small piece of code from a textbook or lab, changing it from a style that uses only primitive types to a style that uses user defined types. Notice how the code feels before and after the change. Often, function signatures become shorter, and the main logic becomes easier to read.

1. User defined types model real problems.
2. Good type design improves clarity and safety.
3. Records, classes, enums, and variants are common.



5.0 Keyword Touch Vocabulary Meaning

Keyword	Meaning in simple words
Data type	Kind of value and its uses
Primitive type	Basic built in simple type
User defined type	New type made by programmer
Record / struct	Group of named fields together
Class	Type with data and methods
Enumeration (enum)	Type with few named values
Union / variant	Type that has different shapes
Type system	Rules that connect values to types

